

# HelixTranslate

## HelixTranslate

**Verified-model book translation — honest by design, never a silent fallback.**

### Summary

HelixTranslate is a high-performance, Go-based ebook translation platform that translates books between 100+ languages using verified LLM providers, with real-time WebSocket monitoring and a strict no-silent-fallback routing policy that fails loudly rather than degrade quietly.

### Short description

Go-based universal ebook translation toolkit. Translates FB2, EPUB, TXT, HTML, PDF, and DOCX across 100+ languages using the strongest verified LLM (via the LLMsVerifier bridge), with REST/HTTP-3 and gRPC APIs, distributed processing, and a real-time WebSocket monitoring dashboard.

### Long description

HelixTranslate is an enterprise-grade, Go-based system for translating whole books between languages using LLM providers — not paragraphs or snippets, but book-length works, end to end. It parses and regenerates multiple ebook formats (FB2, EPUB, TXT, HTML, PDF, DOCX), supports 100+ languages with auto-detection, and offers both CLI tools and API servers (REST over HTTP/3, gRPC, and a WebSocket event stream) so it fits equally into a terminal workflow or a service mesh. Its defining characteristic is *how it chooses a model*: rather than hardcoding a provider and hoping it stays healthy, HelixTranslate delegates all model authority to the LLMsVerifier bridge (pkg/bridge), which selects the strongest *verified* API model and hands back a deterministic, score-ordered fallback chain. Model eligibility is

decided by a weighted score across responsiveness, code, feature richness, and reliability — so the model doing your translation earned its place by proving it works, not by appearing in a config file.

Crucially, the system enforces a “no silent fallback” mandate directly in code: if no provider API key is present, or an operator explicitly requests an unavailable provider, the pipeline returns an honest hard error instead of quietly switching providers or dropping to a local runtime and pretending everything is fine — a rule pinned down by a dedicated pre-build gate and a paired mutation test. Local runtimes (Ollama, llama.cpp) were deliberately removed from the default path so a weaker engine can never silently stand in for a verified one. Around the translation core sits a real-time WebSocket monitoring subsystem: the translation CLI emits typed events to a monitoring server that drives a live web dashboard, while remote SSH workers fan the workload out for distributed translation. Layered on top are multi-pass polishing for consistency, a preparation-phase quality analysis, translation caching to tame cost over long inputs, and vision-driven QA. The whole platform answers to an anti-bluff engineering constitution: tests must prove real, user-visible outcomes, backed by mandatory mutation testing rather than green checkmarks that prove nothing.

## **Why we built it**

To translate long-form books reliably and *honestly* — never shipping a “degraded-but-present” translation. The design premise is that a missing or unverifiable translation must be a loud, hard error, and that model selection must always resolve to a genuinely verified provider rather than a hardcoded guess or a silent local fallback.

## **Why it’s a game-changer**

Most LLM translation pipelines fail silently — they quietly fall back to a weaker model, slip down to a local runtime, or emit partial output while the test suite still glows green and nobody notices the quality cliff. HelixTranslate makes that entire failure mode structurally impossible: model choice is verification-gated, the fallback chain is deterministic and fully transparent, and “no key / no verified model” resolves to an honest hard error rather than a silent shrug. That single design decision turns “did this translation actually run on a capable, verified model?” from a hope you can’t check into a guarantee the system enforces on your behalf.

## What's innovative

- **Verification-gated model routing** via the LLMsVerifier bridge — the strongest *verified* model is auto-selected, so operators state intent, not vendor names, and never hand-pick a provider that might be down.
- **No-silent-fallback guarantee enforced in code** — four explicit routing arms (mock / explicit-verifier / explicit-provider / bridge-default), each of which hard-errors rather than silently switching, plus the deliberate removal of local runtimes from the default path so there is nothing weaker to fall back to.
- **Mechanical enforcement** — a CM-NO-LOCAL-RUNTIME pre-build gate plus a paired mutation test asserts, at build time, that no local-runtime client is ever constructed on the default path: the guarantee can't rot because the build fails if it does.
- **Deterministic, score-ordered fallback chain** — provider-to-provider failover among *verified* models is allowed and fully transparent, a principled distinction from the forbidden silent fallback: you always know which capable model caught the work.
- **Real-time WebSocket monitoring** — typed translation events streamed live to a dashboard, with distributed SSH workers so a book-length job is visible and parallel, not a black box.
- **Anti-bluff testing regime** — mutation testing, negative assertions, real-system runs, and vision-driven QA together ensure "tests pass" can never quietly mask "feature doesn't actually work."

## Biggest technical challenges & how we solved them

- **Guaranteeing an honest translation pipeline (no silent degradation).** Solved by centralizing all model authority in the LLMsVerifier bridge so there's a single decision point to police, encoding four explicit routing branches that each fail loud instead of guessing, ripping local-runtime fallbacks out of the default path entirely, and welding the rule in place with a build gate plus a mutation test that fails the build if the guarantee is ever removed.
- **"Green tests, broken features."** The constitution names this failure mode outright and defeats it with the Anti-Bluff Testing regime: concrete user-visible assertions instead of implementation trivia, real systems in the loop (mocks confined to unit tests), mandatory mutation testing (deliberately break the feature and the test *must* go red), and vision-verified QA that actually looks at the output.

- **Long-form, multi-format quality.** Book-length inputs strain both consistency and budget; solved with multi-pass polishing that revisits the text, a preparation-phase analysis that sizes the job up front, and translation caching that avoids paying twice for the same passage.

## Tech stack

- **Go** — chosen for its concurrency primitives, which map naturally onto parsing, translating, and streaming many chapters at once; the high-concurrency backend, module `digital.vasic.translator`.
- **Gin** — chosen as a fast, minimal HTTP router to serve the REST API surface.
- **QUIC / HTTP/3 (quic-go)** — chosen to give the REST API a low-latency, modern transport that holds up over imperfect networks.
- **gRPC + Protocol Buffers** — chosen for a strongly-typed, high-performance service interface running alongside REST for programmatic callers.
- **Gorilla WebSocket** — chosen to carry the real-time, typed translation-event stream that feeds the monitoring dashboard live.
- **PostgreSQL, SQLite, Redis** — a deliberate three-tier split: PostgreSQL for durable relational data, SQLite for embedded/local state (it also backs the bridge's verified-models store, `data/verified_models.db`), and Redis as the hot cache.
- **unidoc/unioffice + unipdf** — chosen to handle the hard formats: DOCX and PDF parsing and regeneration so multi-format ebooks round-trip faithfully.
- **Cobra** — chosen as the CLI framework powering the unified-translator and its companion tools.
- **golang-jwt (JWT HS256)** — chosen for stateless API authentication, paired with per-IP token-bucket rate limiting and TLS/QUIC transport security to keep the surface hardened.
- **LLMsVerifier bridge (pkg/bridge)** — the linchpin: sources the strongest verified model plus its deterministic fallback chain, and serves as the single enforcement point for the no-silent-fallback guarantee.
- **Testify** — chosen for the Go test suite, including the dedicated `provider_routing_test.go` and the mutation gates that keep the honesty rules honest.
- **Docker / Podman (rootless) + Compose** — chosen for containerized, distributed deployment (`docker-compose.distributed.yml`), with rootless Podman for a tighter security posture.

## Status & honesty notes

- **Status: beta.** Functional platform; the version is stated inconsistently across VERSION/Makefile/AGENTS.md, so it is treated as unsettled.
- **License: TBD.** The README claims MIT but this is not confirmed against a LICENSE file — verify before stating.
- The dashboard/monitor endpoints are localhost-only, not public. WebSocket performance numbers in the docs are stated targets, not verified. ARCHITECTURE.md still lists removed Ollama/local engines (stale).

**Priority tier:** Helix-primary (LLM-infrastructure cluster). Ranks within the Helix platform family, after HelixTrack.